

The Squeeze Loop Strategy:

Catching Coherent-and-Wrong Artifacts with an Author-Independent Executable Oracle

Fabrice Derepas
Canonical

<https://github.com/canonical/squeeze-loop>

June 2026

Abstract

Agentic large language model (LLM) workflows increasingly run in *loops*: one agent produces, another critiques or tests, and the cycle repeats until an acceptance signal fires. They fail in one characteristic way. When that signal is producible — or even merely visible — to the agent being judged, the loop converges on agreement with itself rather than on correctness. We call the result *coherent-and-wrong*: an artifact that passes its own checks while missing the intended property. Self-preference, sycophancy, and tests tuned to the code are all this shape.

The *squeeze loop strategy* removes that failure’s root condition by construction wherever an author-independent executable oracle exists. Every actor is pinned between a soft *upper bound* — a citable authority, the strongest claim it may make — and a hard, executable *lower bound* it cannot alter; it answers to a *different* authority pair from every other (the *disjointness principle*), and a judging actor never sees the implementation it grades. Where no oracle exists, the loop closes instead by nesting that terminates in human review.

This is a methods paper; stripped of the apparatus, its evidence establishes three things. *First, an LLM’s proof can be made trustworthy by a mechanical check.* Applied unchanged to an external UEFI bootloader (Canonical’s `lace`), a *faithfulness gate* strips every annotation the agent added and demands the original source back byte-for-byte — a cheated proof cannot survive — and under it the loop proved 25 functions robust and found 4 real security bugs, with no human judgment and no detection rate to tune. *Second, when a spec genuinely contradicts itself, a smarter model does not help and can hurt.* On a balanced authority conflict the error *rose* with model size ($0.54 \rightarrow 1.00$), every model-based check (self-critique, a second model, a different provider) reproduced the same wrong reading from shared priors, and only an executable ground truth caught it. *Third, the mechanism the strategy is built around — hiding the implementation from the judge so it cannot anchor — does not measurably matter:* across 504 trials its outcome effect never appeared, because a competent model handed an explicit authority simply follows it. The unifying claim those support: what catches a fluent-but-wrong artifact is an *independent executable check* — not a smarter model, not more agents, not hiding the code from the judge. Four executable seeded instances over three terrains (analytics and Rocq proofs, a refund agent, an API guard) demonstrate that the construction instantiates and its gate fires on each — existence demonstrations, not efficacy measurements, so a powered efficacy comparison remains future work.

1 Introduction

The unit of progress in applied LLM systems is shifting from the single completion to the *loop*: plan–act–observe cycles [45, 48], generate–critique–refine cycles [31, 43], and multi-agent pipelines in which differentiated roles (programmer, tester, reviewer, product manager) pass

artifacts to one another until the work is declared done [20, 29, 39, 46]. The promise is division of labor; the recurring disappointment is a particular failure shape. When the signal that terminates the loop can be produced, edited, or simply *seen* by the actor whose work it judges, the loop converges not on correctness but on agreement with itself. The empirical literature documents several faces of this failure: intrinsic self-correction that does not correct [21], evaluators that recognize and favor their own generations [36], judges with systematic position and verbosity biases [49], and models that yield to the user’s framing rather than the evidence [42]. At the level of a whole workflow, this is Goodhart’s law in organizational form [1, 28, 32]: the acceptance signal is a proxy. Any actor able to influence its own proxy will, under optimization pressure, satisfy the proxy rather than the target.

This paper describes a strategy for operating a set of agents so that the failure shape above is excluded *by construction* on the executable side rather than discouraged by instruction. We call it the *squeeze loop strategy*. We scope the phrase *by construction*: against the hard, executable lower bound the failure’s root condition is removed structurally, with no judge in the loop; the soft, editorial side carries no such mechanical guarantee at a single level and is instead closed by *nesting* — an outer squeeze holding a disjoint (U, L) pair over the inner coordinator — terminating in a human or external reviewer (§3). Its two central ideas are simple to state:

1. **The squeeze.** Every actor is held between two constraints it does not control: an *upper bound* — a citable normative authority fixing the strongest claim the actor may make — and a *lower bound* — an executable ground truth whose verdict the actor cannot alter. A deliverable is admissible only in the band between the two; the actor’s freedom is exactly the width of that band.
2. **Disjointness.** Every actor answers to a *different* pair of authorities, and the pairs are chosen so that their intersection is the correct deliverable while no single actor’s evidence base is sufficient on its own. Each actor’s characteristic failure mode is therefore caught by a different actor that cannot share its blind spot; correctness is the fixed point that satisfies all squeezes simultaneously.

The *loop* is the protocol that circulates work through the band: forward through plan documents under a status handshake (DRAFT → editorial APPROVED → DONE), backward through *gap documents* that reopen an item on any divergence, with *done* defined exclusively by machine-checked gates — never by an agent’s report of success.

A concrete case makes the shape vivid. A revenue metric is to be computed, and the handbook mandates *net* revenue (gross minus refunds). An implementer instead computes *gross*: a single, internally consistent program that runs, returns a plausible number, and passes every check it can see, including its own tests. That is coherent-and-wrong: fluent, self-certifying, off. The squeeze catches it not with a better test but by *who holds what*. An independent exerciser, given only the handbook and never the implementation, derives the *net* figure; the two disagree at the acceptance gate. The implementer cannot relieve the constraint, never having held the exerciser’s authority; the exerciser cannot be fooled by the same blind spot, never having seen the gross-revenue code. We return to this instance, measured, in Section 5.

The strategy was developed and operated, not designed on paper. It emerged from running coordinator–worker loops of LLM coding agents, and we instantiate it here as four *executable, reproducible* instances over three *epistemic terrains*: one where an external specification already says what correct is (carrying two instances: tabular analytics and machine-checked proofs); one where no external authority exists for the property at all; and one where the meaning of every construct splits across two planes with separate authorities. The topology stays fixed while the squeeze is rebuilt from different materials each time, every number from a re-runnable harness (Section 5). This is a methods paper: it contributes the pattern and four *existence demonstrations* — single end-to-end runs showing the construction instantiates and its gate

fires on each terrain, not measurements of how often it helps — not a comparative benchmark. A controlled comparison against the pipelines we critique is operationalized and run as an *exploratory* study (Section 8), where the squeeze’s executable oracle alone ships none of the underspecified cases coherent-and-wrong — the live-model failure mode promoted to Section 6. A powered version remains future work, the natural-bug route having met a near-zero implementer error rate.

Contributions. We lead with what the evidence supports, and are explicit that the empirically load-bearing object is the executable oracle, not the apparatus around it.

(i) **A faithfulness gate that makes proofs trustworthy by construction.** Applied *unchanged* to an external UEFI (Unified Extensible Firmware Interface) bootloader the authors did not write (Canonical’s *lace*), a disjoint *strip-and-recompile* gate rejects any fabricated proof (code edited to pass cannot survive); under it the loop proved 25 input-facing functions robust and surfaced 4 real security defects. The squeeze-specific part is the gate; *finding* the defects is what the underlying verifier delivers (Section 4).

(ii) **A live failure mode that only an executable oracle catches.** A capable agent fills a contradictory or underspecified soft spec with its own *coherent* reading that diverges from the answer key — with no *monotone* improvement as capability rises, and, on the genuinely-balanced GDPR-erasure “survivorship” fork, error that *rises* with model size ($0.54 \rightarrow 1.00$): scaling makes the contradiction *worse*, not better. Every model-based check (self-critique, same-context critics, same- and cross-provider casts) reproduces the model-consensus reading; only the executable lower bound catches it, a floor reproduced across a qwen capability ladder (4B–27B) and a three-provider panel (Section 6).

(iii) **A clean negative result on the context barrier.** The strategy’s named differentiator is disjointness of evidence enforced by a physical context barrier; yet a pre-specified isolation experiment found the barrier’s *outcome* effect undetectable in every condition we could cleanly test (504 draws, three regimes, two provider families). We report this honestly: empirically the independent executable oracle, not context disjointness, carries the load; the barrier remains a *structural* guarantee (anchoring impossible in principle) we could not make bind on competent models (Sections 3.4, 8, 9).

(iv) **The squeeze loop as an organizing frame.** The construction — every actor pinned between a soft upper bound and a hard, executable lower bound, answering to a disjoint authority pair — instantiated as four executable, reproducible instances over three terrain archetypes (Sections 3, 5), with anti-collapse stabilizers (Appendix B), an evaluation protocol (Section 8), and a reflexive case study in which the paper’s own process caught 8 defects of varying severity — from structural coherent-and-wrong errors to editorial over-claims — in its own claims, an $n=1$ illustration (Section 7).

Scope (what we claim, and what we do not). We state our scope plainly. We claim the construction is precise and its wiring correct, recurring across 4 executable instances over 3 terrain archetypes whose gates catch every *seeded* coherent-and-wrong artifact — these establish *existence* of the construction, not efficacy — and that it surfaced 4 *natural* such defects in an external codebase (Section 4); the efficacy weight rests on the two live findings below. The central claim is a *structural guarantee* about *possibility*, not a frequency: the barrier removes the *contextual* producibility of the acceptance signal — a judge cannot see or anchor to the artifact it grades — but not the correlated error that shared pretraining induces, which survives it (Section 9). The guarantee is also asymmetric: its *hard* side is structural (the executable lower bound is mechanical), but its *soft* side — faithfulness to the interpretation-laden upper bound — is *procedural*, resting on the editorial judgment of Gate A (Section 3.3) and only as strong as that single coordinator judgment. The seeded instances are consistency checks that the wiring fires: the one real-agent barrier-on/off pilot found no measurable effect (the spec-driven

exerciser catches regardless), and on small, well-specified tasks the per-implementation error rate is near zero, so a *powered* efficacy claim remains future work (Section 8). The two live findings above are the load-bearing evidence. We state the resulting irony plainly, since it shapes what the paper establishes: the numbers our harness regenerates byte-for-byte are exactly the seeded checks we do *not* rest weight on, while the two load-bearing findings (the `lace` defects and the contradiction floor) are non-deterministic, unpinned, single-team — reproducible at the level of the *floor*, not the per-draw decimal (Sections 4, 6).

2 State of the Art: Loops, Critics, and Their Failure Modes

We organize prior work by the question that motivates the squeeze loop: *where does the signal that closes the loop come from, and who can touch it?* Each line of work below forges part of the link the squeeze loop makes explicit — between a soft authority (what *should* be true) and an executable ground truth (what *is* the case) — but leaves the per-actor evidence base, the thing that actually carries that link, unspecified.

2.1 Single-agent iterative loops

The simplest loops keep producer and critic in a *single* agent, so the closing signal is fully intrinsic — the case our question is most wary of. ReAct interleaves chain-of-thought reasoning with actions whose observations feed back into the next step [48]; Voyager iterates skill acquisition against an embodied environment [45]. In both, the environment supplies genuine external feedback, an executable lower bound in our vocabulary, and the loops are correspondingly robust; Voyager’s growing skill library is the closest prior analogue to the skill our own squeezed agents accumulate (Section 8). Self-Refine [31] and Reflexion [43] close the loop with the model’s own critique (Reflexion optionally grounding it in test signals). Constitutional AI [5] critiques generations against an explicit normative document, an early example of an *upper bound* as a citable artifact — though in its self-critique stage generator and critic share weights and context. CRITIC [18] grounds critiques in external tools and finds such feedback crucial where purely intrinsic self-critique falls short; process supervision [30] moves the signal from outcomes to verified steps. The common thread is that loop quality tracks the *externality* of the closing signal.

2.2 The self-evaluation problem

When the closing signal is intrinsic, the evidence is cautionary. Huang et al. [21] find that LLMs cannot yet self-correct reasoning without external feedback, and that self-correction can degrade performance. Panickssery et al. [36] show that LLM evaluators recognize their own generations, with self-recognition correlating linearly with self-preference bias: the evaluator is not neutral about authorship. LLM-as-judge setups exhibit further systematic biases [49], and models exhibit sycophancy toward the user’s stated position [42]. Framed generally, an evaluator that shares the generator’s parameters, context, or incentives is a *proxy*, and proxies under optimization pressure invite gaming [1, 28, 32]. The squeeze loop treats this not as a model defect to fine-tune away but as an organizational invariant to design around: no actor may judge its own work, and no actor may hold enough context to fabricate a self-consistent wrong answer.

2.3 Multi-agent pipelines, debate, and oversight

Adding agents distributes the labour but not the *evidence base*: the closing signal stays intrinsic to the team when every actor sees the same things. Role-played multi-agent systems — CAMEL’s communicative agents [29], AutoGen’s conversation programming [46], MetaGPT’s

SOP pipelines [20], ChatDev’s software-company simulation [39] — demonstrate that differentiated roles and structured artifacts improve over single-agent baselines; MetaGPT is closest in spirit, with roles communicating through typed documents rather than free chat. The squeeze loop’s observation is that *role* differentiation is not yet *evidence* differentiation: in most of these systems every agent reads the shared transcript, so the “tester” has seen the implementation and the “reviewer” the tester’s reasoning, and the biases of Section 2.2 re-enter through the shared context. Multi-agent debate [14, 22] has agents argue before a judge; in its LLM realization [14] debaters exchange full reasoning across rounds — deliberately shared context, appropriate for argument-checking but orthogonal to independence of evidence. Scalable-oversight research [8, 9] asks how weaker judges can supervise stronger workers; the squeeze loop is complementary, asking how to *position* judges and workers so the judging signal is not capturable regardless of relative capability. Prover–verifier games [2, 26] are the closest machine-learning neighbour: a strong prover and a trusted verifier are trained against each other, with a *sneaky* prover rewarded for plausible wrong answers [26] — a trained model of our coherent-and-wrong — and the capability gap is varied as our cross-model cast is. The difference is the locus of enforcement: they obtain checkability by *training* shared-weight networks; the squeeze does so *organizationally*, as an information-flow invariant over each agent’s context, so it applies to frozen models and to the terrain where the soft-to-hard link is hardest to forge: a normative soft truth with no trainable verifier and no external oracle. A recent multi-agent-LLM failure taxonomy [10], which counts weak verification and inter-agent misalignment among its three top failure categories, is the backdrop the squeeze is built against.

2.4 Verifier-in-the-loop code generation

Software engineering supplies the clearest executable lower bounds. SWE-bench and SWE-agent [24, 47] use held-out tests as ground truth; AlphaCodium [40] structures generation as a flow against public and AI-generated tests; LLM-Modulo frameworks [25] argue for pairing LLM generation with external sound verifiers. Clover [44] closes the loop by checking consistency among code, docstrings, and formal annotations, on the hypothesis that passing all checks implies correctness, reporting no false positives on its CloverBench Dafny suite. Deductive platforms such as Why3 [16] make the strongest lower bounds available: a proof obligation either discharges or it does not. Yet even with a sound verifier in the loop, a workflow can converge on a result *valid but answering the wrong question* unless the property’s author and its exerciser are independent of the implementer — the gap the squeeze loop closes.

2.5 Classical antecedents outside machine learning

Every load-bearing idea has an ancestor; what is new is the setting — these antecedents drew independence from separate *people*, while the squeeze must construct it among agents sharing one model’s priors. N-version programming [4] and the Knight–Leveson result [27] — independently written versions fail on *correlated* inputs — are the lineage of our central limitation: barriers remove conversational coupling, not shared-pretraining correlation (Section 9). Author–inspector separation (Fagan inspections [15]; Cleanroom [35]) is the pre-LLM statement of “the implementer never touches the acceptance evidence”; executable specifications (TDD [7], Design by Contract [34]) and separation of duties (Clark–Wilson [11], Saltzer–Schroeder [41]) supply the rest, under a Popperian stance [38] — an artifact earns trust by surviving refutation authored by another. Mutation [12, 23] and differential testing [33] are the verifier calibration and the implementer/exerciser-agreement principle we apply (Section 5).

Table 1: The squeeze loop’s four design points vs. representative systems. $\checkmark$ present; $(\checkmark)$ partial; $\checkmark^\circ$ *structural property present but no measured outcome effect* (§8); — absent. *Disj.*: pairwise-distinct authority pairs; *Barrier*: physically enforced context separation; *Gate*: *done* defined only by machine gates; *No-auth*: *having* a construction for when no external authority exists (Archetype B), not an oracle-backed guarantee, which B lacks (§5). The crucial honesty is the Barrier column: the squeeze’s mark is $\checkmark^\circ$ — the separation is structurally present, but a pre-specified experiment found no effect on outcomes for capable models.

System	Disj.	Barrier	Gate	No-auth
MetaGPT [20]	—	—	$(\checkmark)$	—
ChatDev [39]	—	—	—	—
Multi-agent debate [14]	—	—	—	—
AlphaCodium [40]	—	—	$(\checkmark)$	—
Clover [44]	$(\checkmark)$	—	$\checkmark$	—
Prover-verifier games [26]	$(\checkmark)$	—	$\checkmark$	—
Squeeze loop (this paper)	$\checkmark$	$\checkmark^\circ$	$\checkmark$	$\checkmark$

2.6 The gap

Across these literatures, roles are assigned, verifiers are added, and artifacts are structured — but the *per-actor evidence base* is rarely an explicit design object. Prior systems seldom (i) require the (upper, lower) authority pairs to be pairwise distinct, (ii) enforce information barriers physically (by what is in each agent’s context) rather than honorarily (by instruction), (iii) define *done* exclusively by gates, or (iv) say what to do when no external authority exists for the property being built. The squeeze loop strategy is an organizational layer addressing exactly these four points; it composes with, rather than replaces, the loops surveyed above. Against the closest antecedents, three pieces are genuinely new. The central one is the disjointness of authority *pairs*: every actor answers to a *different* (U, L) pair, not the single author–inspector split of Fagan inspections or IV&V — pairwise-distinct pairs make correctness the *intersection* of several bands, no one of which suffices, so each actor’s blind spot falls inside another’s evidence base, which a lone author–inspector split cannot provide however rigorous. The second is *physical context barriers* specific to LLM agents: where Cleanroom forbids developers from running their own code, here the implementation is absent from the judge’s context entirely. The third is an explicit construction for the *no-external-authority* case (Archetype B), which document-centric pipelines such as MetaGPT leave open — with the trade-off, stated plainly here and not only in Limitations, that this is also the *weakest* of the three: with no external oracle its lower bound (an authored archive) shares the property author’s provenance, so its soundness reduces to author independence, the classical Fagan/Cleanroom move (§5, §9). The novelty is the explicit *construction* for the case, not a stronger guarantee within it. Table 1 places the four points against representative systems; prover-verifier games, the closest antecedent, train for checkability rather than enforcing it by context disjointness over fixed models.

We position our evaluation methodology against the *applicable formal methods* manifesto [17]. The squeeze loop is not itself a formal method but an organizational layer that *wraps* them, so we read its principles as a lens, not a membership test. We follow its *Evaluation* principle — demonstrate applicability credibly with representative examples and tools usable by others, disclosing benefits *and* foreseen barriers: the external **lace** application is a *case study* in the manifesto’s sense, the four seeded instances are *existence demonstrations*, both with re-runnable harnesses (§4, §5), and the Limitations (§9) supply the foreseen-barriers half. We conform only *partially*: the manifesto’s *Usefulness* principle asks for effectiveness against a non-formal alternative, which we report as exploratory and not yet powered (§8).

Term	Meaning (and where defined)
Coherent-and-wrong	An artifact that passes its own checks while missing the intended property (§1).
Squeeze	A single actor pinned between an upper and a lower bound (Def. 1).
Upper bound (U)	A <i>soft</i> , citable normative authority — the strongest claim an actor may make.
Lower bound (L)	A <i>hard</i> , executable ground truth whose verdict no actor can alter.
In-band	A deliverable faithful to U and consistent with L .
Hard / soft truth	Executable (mechanical, interpretation-free) vs. normative (interpretation-laden).
Disjointness principle	Each actor answers to a <i>different</i> (U, L) pair; disjointness of <i>evidence</i> (the implementation is absent from the judge’s context), not of model weights (§3.4).
Canonical cast	The five roles — coordinator, property author, implementer, exerciser, probe (Table 3).
Gate A / B / C	Editorial approval / machine-checked acceptance / coverage (the <i>no-blend</i> rule — one plane’s check may not stand in for another’s — and the coherent-and-wrong guard) — the per-item gates (§3.3).
Gate S	Skill-consistency: a monitor squeezes a learned skill against U and L (§3.3).
Terrain archetypes	A transcription (external spec); B authored authority (no external spec); C split planes (§5).

Table 2: Glossary of the load-bearing terms, with where each is defined.

3 The Squeeze Loop Strategy

The introduction argued that a loop fails when its closing signal is producible by the agent being evaluated. The empirically load-bearing element of the fix is the *executable lower bound*: a judge holding an independent oracle catches the coherent-and-wrong artifact (Section 6). The construction adds a structural guarantee on top — remove the artifact from a judge’s context so it cannot anchor to what it grades — though that barrier’s *outcome* effect we could not demonstrate on competent models (Section 8); disjointness of *evidence*, not of weights, is the principle it enforces, *not* model diversity. This section gives the construction in four steps: the atomic relation bracketing a *single* actor between two bounds (*The squeeze*); the *cast* a real task needs; how they are wired into a loop whose *gates* decide done; and the condition that makes the wiring sound (*The disjointness principle*). The archetypes, stabilizers, case studies, and the paper’s own self-application are all this construction instantiated. Table 2 collects the load-bearing terms and where each is defined.

3.1 The squeeze

We start with the atomic relation, before any loop is built around it. A single actor is pinned between two bounds, stated precisely so the compliance conditions, the cast, and the loop can all be expressed in its terms.

Definition 1 (Squeeze). *An actor a_i is squeezed by an authority pair $\pi_i = (U_i, L_i)$ where U_i , the upper bound, is a citable normative artifact fixing the strongest claim a_i may make, and L_i , the lower bound, is an executable ground truth — a runnable process whose verdict a_i cannot alter. A deliverable d is in-band for a_i iff d is faithful to U_i (it claims no more than U_i licenses) and consistent with L_i (executing L_i does not contradict it). Anything above the band is over-claiming or improvisation; anything below it is unfaithfulness to reality.*

Remark 1 (Hard truth and soft truth). *The two bounds are both sources of truth, but of opposite kind, and the squeeze binds one to the other. The lower bound L_i is a hard truth: an executable process whose verdict is mechanical and interpretation-free — a test passes or fails,*

a number recomputes or it does not, a proof obligation discharges or it does not. The upper bound U_i is a soft truth: a norm, standard, or specification in natural language, authoritative but interpretation-laden, since the same English that states a problem precisely still admits more than one faithful reading. A deliverable is in-band exactly when it is a reading of the soft truth that the hard truth does not refute; coherent-and-wrong is then precisely a plausible interpretation of the soft upper bound that the hard lower bound contradicts (or, dually, an artifact satisfying the hard check while betraying the soft intent). Neither bound suffices alone: the hard truth cannot say what ought to be computed, the soft truth cannot say what is. The editorial Gate A (the coordinator’s approval by judgment) adjudicates against the soft truth, the machine Gate B against the hard truth — both per-item gates defined with the loop in §3.3. What changes across the terrain archetypes of Section 5 is only how hard the available lower bound is and how soft the governing upper bound; the binding between them is the invariant the strategy preserves.

Definition 2 (Squeeze-loop compliance). A workflow over actors $a_1, \dots, a_n$ with pairs $\pi_1, \dots, \pi_n$ is squeeze-loop compliant when:

- (C1) **Disjointness.** The pairs are pairwise distinct, $\pi_i \neq \pi_j$ for $i \neq j$, and chosen so that the intersection of the in-band sets is the correct deliverable while no single π_i suffices to certify it.
- (C2) **Catchability** (a design aspiration, not a proven universal). For each actor a_i , the cast is designed so that the failure modes a_i is known to exhibit, and that survive its own squeeze, are detectable from some other pair π_j , $j \neq i$. We do not claim this for every conceivable failure — our experiments exercise specific failure classes (the seeded and natural defects below), not the full space.
- (C3) **Physical barriers.** At delegation time, each actor’s context contains exactly its authorities and the public interfaces it must exercise; evidence it is denied (most importantly: the implementation, for the actors who judge it) is absent from its context, not merely off-limits by instruction.
- (C4) **Gate-defined done.** An item is done iff a fixed set of machine-checked gates passes; no actor’s self-report contributes to the definition.

Remark 2. (C1) is the construction’s distinctive design point and the part most often missing from existing pipelines: every actor answers to a different pair of authorities, and the pairs are chosen so their intersection is the correct deliverable while no single actor’s evidence base is sufficient on its own. Under (C1) and, for the failure classes (C2) is designed to cover, a corruption of any single source — or any single actor’s blind spot in those classes — cannot propagate silently, because no other actor reads from it. We are precise about where this carries empirical weight: both live findings land on the executable oracle (Section 6), and disjointness of evidence is load-bearing specifically on the oracle-free terrain (Archetype B), where it is the only defense; elsewhere it is the structural precondition the oracle’s catch builds on, not the thing measured to catch.

3.2 The canonical cast

A single squeeze brackets one actor; a real task needs several. Table 3 gives the five canonical roles with the squeeze each one carries. Every role also carries a *forbidden move*: the role-crossing action that would relieve its own squeeze. The forbidden moves are load-bearing, not hygiene (Appendix B). These role names — “author”, “judge”, and the rest — are organizational descriptions of information flow, not claims about agent cognition.

Actor	Builds	Squeezed from above by	Squeezed from below by	Forbidden move
Coordinator (main thread)	approvals, sequencing, gate verdicts	the binding plan/spec sections (but its <i>editorial</i> judgment, Gate A, is itself un-squeezed within the loop; §3)	the gates' machine output	editing source; approving a plan it has not editorially judged
Property author (upstream)	the precise property spec: enumerated obligation clauses, the one attack the negative must catch, the explicit NOT-claims	the external category or flagship use case	expressibility: each clause must be dischargeable by <i>some</i> mechanism	proposing the implementation; reading its code
Implementer	the item inside the system, plus its documentation	the spec's strongest mandated claim	shipped machinery, laws, a <i>standing invariant</i> (a property re-checked after every item, defined in §3.3)	weakening a clause or gate to land; touching the acceptance evidence
Exerciser	the positive/negative acceptance evidence	the spec's acceptance clauses and the documented surface <i>only</i>	what actually passes or proves when run	reading the implementation or any diff
Probe	minimal experiments with recorded verdicts	the design claim under test (one claim per probe)	the tool's <i>actual</i> behaviour	implementing anything; a probe writes a verdict, never a fix

Table 3: The canonical cast and its squeezes. A new role is justified only by a new evidence base, never by workload: two actors with identical (U, L) pairs are one actor.

3.3 Loop mechanics and gates

With the cast fixed, we wire it into a running loop whose *done* is decided by gates, never an actor's self-report. Work circulates through *paired traceability documents*: a forward plan **spec- N** opened at **STATUS: DRAFT**, set to **APPROVED** only after the coordinator's *editorial* judgment (amend, cut, sharpen — never a binary stamp), and to **DONE** only when the gates pass; and a backward **gap- N** document, written by whichever actor observed a divergence, carrying the same number as the plan that answers it. No source edit ever happens from a **DRAFT**: the plan is the higher-risk half, judged before code exists. Author separation is physical — the exerciser's context contains the documented surface and acceptance criteria only; to learn how the implementation behaves, it runs it and reads the verdict, never the diff.

Three gates structure each item (Figure 1). Gate A validates the property or plan against the upper bound. One honest asymmetry belongs here, not only in Limitations: a single loop's Gate A is the coordinator's *editorial* judgment, and within that loop it is the one actor nothing squeezes — no disjoint (U, L) pair is held over it. The soft half would then funnel through an un-squeezed judge. The construction's answer is to *nest*: wrap the loop in an outer squeeze whose monitor holds a disjoint (U, L) pair over the inner loop's coordinator and its soft outputs — Gate S (below) generalized from one learned skill to the whole loop, the *squeeze-monitoring-a-squeeze* pattern. Nesting *closes* the inner loop's Gate A: that coordinator is now squeezed

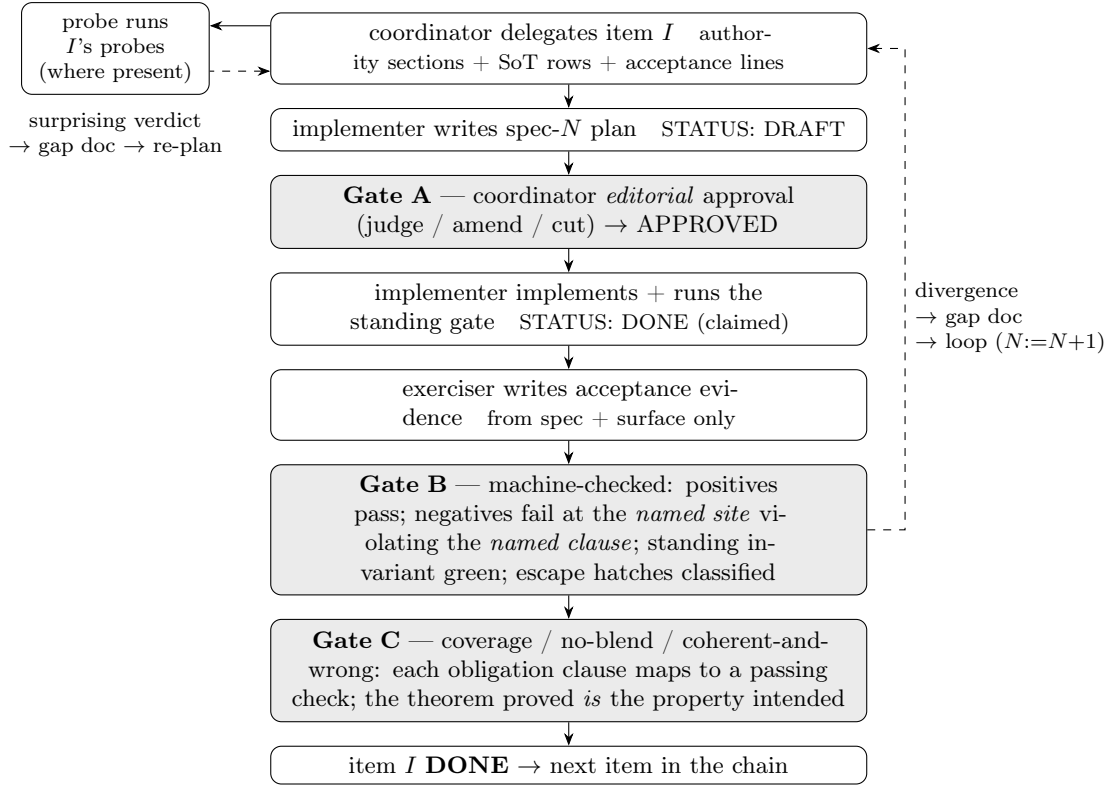


Figure 1: The per-item pipeline. Gate A is the coordinator’s *editorial* approval of the plan (judgment, not machine output); Gate B is machine-checked acceptance; Gate C is the coherent-and-wrong / no-blend coverage guard. Forward motion through the status handshake; backward motion through gap documents; *done* defined only by the gates.

from outside, by an authority it does not hold. Both our applications are built this way — the four seeded instances (Section 5) are squeezed by the outer loop that produced this manuscript (§7), and the `lace` application nests `creusot-sl` inside a monitoring loop, `creusot-monitoring` (§4) — and the recursion *must* terminate in a *human*. This is the whole answer to the regress objection (*who squeezes the squeezer?*): a stack of automated monitors only relocates the unsqueezed editorial judgment one level up — it can never close it. Only a person outside the entire construction can, and the strategy *requires* that one does. A human must end the nesting, full stop (§7 reports exactly this terminal squeeze working: a Gate-A caption error every internal gate missed and an outside reviewer caught). That terminus is bounded, which keeps it both tractable and disjoint: the human does not re-verify the artifact from scratch — the hard gates have already settled the executable side — but adjudicates only the soft-side residual, the *delta* between what the upper bound claims and what the lower bound established, holding *U* and the gate record without having produced the work they judge. So the soft side is closed by disjoint authority all the way up to that mandatory human terminus; the *hard* half (Gate B/C against the executable lower bound) is closed mechanically, *by construction*, with no judge in the loop at all. Gate B is machine-checked acceptance, including a *standing invariant* rerun after every item — in our instances *total additivity*: byte-identical output for every pre-existing input, re-confirmed and determinism-checked — which turns regression into a definitional impossibility (“if your change alters an existing output, your change is wrong by definition”). Gate C is the coverage guard against *coherent-and-wrong*: each obligation clause maps to a specific passing check, and the theorem actually proved is checked to be the property intended. Where external ground truth exists Gate C is machine-checked; where it does not it is defended by author independence (Section 5).

A fourth gate operates on a different cadence. **Gate S** (*skill consistency*) fires whenever the loop *accumulates* a learned skill — a soft heuristic an actor consolidates from its own caught errors. Because that actor shares the skill’s blind spot, the skill is checked by a *monitor squeeze* holding a disjoint evidence base (the loop’s own upper bound and its executable lower bound used as an oracle), which returns *pass*, a *carve-out* (a narrowing exception that defers to the upper bound where the skill over-reaches), or *reject*. Gate S is the no-blend rule of Section 3.3 applied to a loop’s own learned outputs — the per-skill case of the nesting just described, a squeeze monitoring a squeeze, demonstrated deterministically in Section 8.

3.4 The disjointness principle, operationally

Compliance condition (C1) becomes an artifact: the *squeeze table* (Table 3 for the case at hand), written by the coordinator *before any delegation* and pasted row-by-row into each prompt. An actor learns, from fresh context, which authorities bind it and, by omission, which evidence it is denied. The disjointness this enforces is of *evidence*, not of model weights: the exerciser is independent because the implementation is absent from its context and it holds an executable oracle, not because it runs on different parameters. Two actors on the same model but different evidence bases are disjoint here; the same model judging the same context is not. Section 6 measures that same- and same-provider cross-model casts ship identical errors, so — in the only family we could test — it is the independent executable oracle, not model diversity, that carries the gate (a single cross-provider model corroborates the shared blind spot while partially decorrelating; a powered multi-provider replication remains owed, §9). Disjointness and the oracle do two distinct jobs, and we keep them distinct: disjointness of *evidence* removes the *possibility* of anchoring — the structural guarantee, and the only defense where no oracle exists (Archetype B) — while the *executable lower bound* is what *catches the residual* correlated error in the cases we measured. The live finding of §6 speaks to the second: where an oracle exists, model-disjointness alone does not catch the shared blind spot; the oracle does. The structural precondition (disjointness) and the catcher (the oracle) are complementary, not competing. Balance then comes from asymmetry: the implementer can produce something coherent-and-wrong, but the exerciser, authored from the property alone, cannot be fooled by the same blind spot; the exerciser could write a vacuous test, but Gate B demands the negative fail at a named site violating a named clause; the property author could over-reach, but expressibility-from-below rejects undischARGEABLE claims; the coordinator could be tempted to ship, but the gates are machine output it cannot edit. Correctness is the fixed point satisfying all squeezes at once: the catchability of condition (C2), which Section 5.1 makes measurable rather than asserted.

A machine-checkable representation (SL-1.0). The squeeze table need not stay prose. Each loop serialises to a compact registry in which every source of truth — an upper-bound authority, an executable oracle, a produced artifact, a producer’s private rationale, a human terminus — is declared once with a stable identifier, and every actor names the sources it reads as bounds and the sources it is denied, by identifier. Disjointness then stops being narrated and becomes set algebra a checker runs: no actor’s bound sources may intersect what it produces (self-certification), an actor’s denied set must be disjoint from its bounds (this checks the *declared* forbidden-set; the barrier’s physical enforcement lives in the prompts, not the registry), and every characteristic blind spot must be *assigned a different catcher* (condition (C2)). The load-bearing subtlety survives the encoding: a judge may *run* an oracle over an artifact it is barred from — that is how it learns the implementation’s behaviour without reading its internals — but it may never list that artifact among its bounds. The same registry renders to a labelled diagram (Figure 2) and is the artifact an auditor or a monitoring loop reads. What a green check certifies is bounded: that the *declared* authorities are pairwise disjoint — set algebra over the registry, not a verdict that the registry faithfully describes the delegation contexts it summarises. The registry is a claim, distrusted like an org chart and checked against the actual prompts, not the

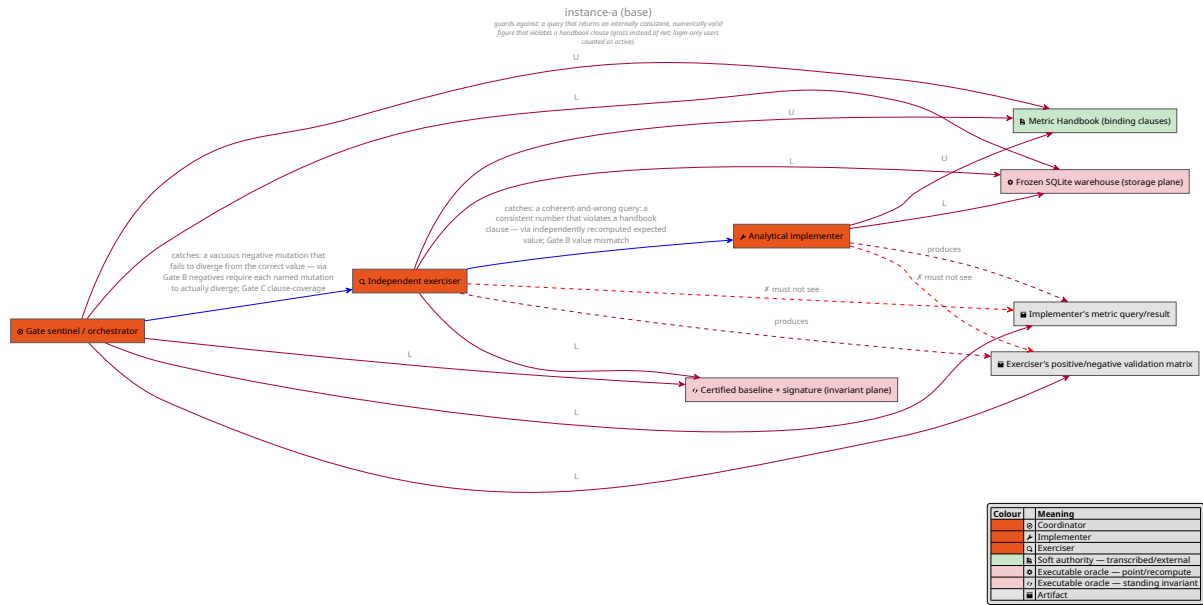


Figure 2: The SL-1.0 rendering of the tabular-analytics base loop (Archetype A, §5): each actor (orange) is squeezed between its upper-bound authority (green, the metric handbook) and its executable lower bounds (pink, the frozen warehouse and the standing-invariant ledger); dashed edges are the physical *must-not-see* barriers, and the blue catchability edges name which coherent-and-wrong each actor catches. Generated from the loop’s *.sl.json by one renderer shared across every instance in this paper; the disjointness checker reads the same file.

territory. The check is a floor, not a ceiling along the other axis too: a green verdict certifies that the loop’s *authorities are disjoint as declared*, never that any deliverable is *done* — gate-defined done (condition (C4)) stays deliberately out of the registry, in the gate evidence.

4 A Real-World Application: Trustworthy Proofs (and Real Bugs) in a UEFI Bootloader

We open with the paper’s headline real-world result: the strategy applied *unchanged* to an *external* codebase the authors did not write — Canonical’s **lace** UEFI bootloader framework. The loop proved 25 input-facing functions robust and surfaced 4 real, reproducible security defects on a pre-OS attack surface — *natural* coherent-and-wrong in production code, direct evidence the target failure occurs in the wild. The squeeze-specific contribution here is *trustworthy proofs*: the disjoint faithfulness gate makes a proof trustworthy *by construction*, so a fabricated “proof” (code quietly edited to pass) cannot survive its strip-and-recompile check (§4.1). *Finding* the defects is a result the underlying deductive verifier delivers, not a squeeze-specific one. This is a *case study*, not the controlled comparison of Section 8; read it for existence and reach, not an efficacy rate. The construction is the one already defined (the cast and gates of Section 3), instantiating the terrain archetypes formalized next (Section 5). The complete artifacts, issue reports, and per-function verdicts are in the supplementary material under **real-use-case/**.

4.1 Materials and the two loops

Rust and Creusot. Rust’s ownership discipline rules out use-after-free and data races in safe code, but *not* the failure classes that matter for a parser of hostile input: integer overflow (a panic in debug, a *silent wrap* in release), out-of-bounds indexing, unsigned subtraction underflow, and unbounded recursion (a native stack overflow). In a bootloader — pre-OS, no sandbox, parsing

data it does not control (a PE (Portable Executable) image on the EFI System Partition (ESP), a partition table on any disk, a `grub.cfg` on any scanned filesystem) — each is a pre-Secure-Boot-measurement denial of service (DoS), or on a release-mode silent wrap a wrong-length or wrong-sector hazard in the loader path. The executable lower bound is **Creusot** [13], a deductive verifier translating Rust through Why3 [16] to SMT (satisfiability-modulo-theories) solvers, with specifications in its Pearlite contract language. The single move that turns it into a bug-finder is to *specify a parser for total robustness*: no panic, overflow, or out-of-bounds read for *any* input, with no input-restricting precondition. An unproved verification condition is then not a proof failure but a pointer at a real, reachable bug, and the guard that discharges it is the fix.

The two nested squeeze loops. The application is built as two loops instantiating the strategy of Section 3. *creusot-sl* verifies one file (Figure 4): its upper bound is an external requirement made precise as Pearlite, its lower bound is *cargo creusot* discharge plus tests plus a mutation probe, and it runs the canonical cast with the implementer (Table 3) recast as an *annotator* that adds proof scaffolding but never rewrites behaviour. *creusot-monitoring* verifies a whole crate by driving a *creusot-sl* sub-loop per file and monitoring its soft outputs: a loop monitoring a loop (Figure 5), the Gate S pattern of Section 3.3, with the mission guidance as upper bound and *creusot-sl*’s machine verdicts as lower bound. “Applied unchanged” means the topology and squeeze are fixed while the gate *materials* are rebuilt per terrain. Two specializations carry the soundness. A *faithfulness gate* re-derives the barrier mechanically: stripping every annotation must reproduce the *byte-identical* original source (we verify *lace*, we do not rewrite it), so a fabricated proof is caught and reverted automatically. What makes this the squeeze rather than ordinary annotation hygiene is *who* enforces it: not the prover choosing not to edit the code (a discipline it can lapse from, or be prompt-injected out of), but a disjoint mechanical check the prover cannot influence and that runs regardless — the producer of a proof is structurally barred from also certifying its faithfulness. A security *Gate C* rejects any `#[requires]` that narrows the adversarial input domain — the dominant coherent-and-wrong being a green proof that quietly *assumed the attacker away*, worthless when malformed input is the whole point. As in the constructed instances the barrier is physical: each *creusot-sl* runs as a sub-agent, so a judge sees only returned verdicts, never a producer’s rationale.

4.2 Results

The loop was first hardened, unsupervised, on a *warm-up corpus* — Creusot’s own test suite, stripped of all annotations and reduced to the 241 files that compile as plain Rust — which it re-annotated and proved (74 proved, 8 partial, 6 failed, 153 trivial), confirming the loop runs at crate scale before it was pointed at *lace*.

This is where the squeeze, not Creusot, earns its keep. The *lace* defects below surfaced as Creusot verification conditions a lone agent running adversarial total-robustness specs might also have found; *finding* them is a Creusot result, not a squeeze-specific one. What the squeeze adds is that the proofs are *trustworthy by construction*. An agent that “proves” code by quietly editing it is the coherent-and-wrong failure exactly, and only a disjoint check catches it: the faithfulness gate strips every annotation and requires the original source back byte-for-byte, so such a fabricated proof cannot survive — mechanically, unattended, with no detection rate to tune or report.

Pointed then at the *lace* parsers of untrusted boot-time input, the loop carried every input-facing parser function to a verdict, with this disposition: 25 **robust** (8 on the disk/ESP surface, 17 on the peripheral/firmware surface), 4 *defect-bearing*, 2 *partial*, and ~6 *blocked* by verifier modeling gaps (an approximate count — the blocked files are not cleanly function-decomposed). That is about 36 distinct functions, the recursion defect (**ISSUE-4**) lying inside one of the blocked files and so counted in both the defect-bearing and blocked tallies. Each robust function is provably free of panic, overflow, and out-of-bounds access for *any* input, under totality-only

extern-specifications and with no `#[trusted]` or `assume!` on any verified logic. The loop found 4 real defects, each with a minimal adversarial input and a one-line fix (filed as `ISSUE-1..4` under `real-use-case/lace-issues/`; five distinct hazards, the PE section-alignment site carrying two): a subtraction underflow in the PE parser (`e_lfanew - 64` on a crafted header), a divide-by-zero and a `u32` multiply overflow in PE section alignment, a `u64` multiply overflow in the MBR (Master Boot Record) extended-partition walk, and an unbounded recursion in the GRUB `submenu` parser (a stack-overflow DoS). Of the four, three surfaced exactly as verification conditions that would not discharge once the parser was specified for total robustness; the remaining one — the recursion — was found by call-graph analysis, Creusot being blocked on that file. Two arithmetic defects were reproduced on `rustc` 1.96.0 in both debug and release. Independently, the run re-checked the architecture report’s structural claims and reproduced its `unsafe-site` counts exactly (77 in the platform crate, 5 / 3 / 0 in the others).

4.3 What it does and does not establish

A *robust* verdict buys exactly one thing: that function provably cannot panic, overflow, or read out of bounds on any input, removing it from the exploitable surface. It is *not* a full mathematical proof of the `lace` codebase, and the experiment makes no such claim. Robustness is neither functional correctness nor a security property: Secure Boot enforcement, TPM (Trusted Platform Module) measurement, and correct kernel handoff are firmware/behavioural properties Creusot cannot model, and were routed out-of-scope rather than pretended. The 77 `unsafe` FFI (foreign function interface) sites in the platform crate are outside the loop’s reach by design — *measured* statically (the run reproduced this count exactly), not proven. The ~ 6 *blocked* functions of the disposition above arise from verifier modeling gaps (no `Invariant` for `String`, missing `&str`-iterator specifications, recursive-type rejection), so their safety rests on by-hand argument; the recursion defect itself lives in one such file. The 2 *partial* functions are panic-safe with no counterexample but carry a residual obligation the bundled solvers did not close (SMT incompleteness, not a bug). The proofs rest on a stated trust base: totality-only extern-specs and a hand-built faithful harness whose fidelity the faithfulness gate checks but does not eliminate as an assumption.

Two scope limits bound the contribution itself. First, the 4 defects are real findings, but the *absence* of further defects in the robust functions is a guarantee only for those functions under their trust base, not a clean bill of health for the bootloader. Second, no baseline pipeline was run on the same code, and the run used a single team, provider family, and toolchain. This is an *existence* result, not the powered efficacy comparison of Section 8. What it adds beyond the seeded instances is what those could not show: *natural* coherent-and-wrong defects exist in real, externally-authored code, and a disjoint executable lower bound surfaces them. It approaches the real-bug study of Section 8 from the deductive-robustness side, complementary to the SWE-bench [24] patch-generation route wired there.

Replication. A replicating team needs `rustc` 1.96.0, the Creusot toolchain, and the faithfulness gate (`verify/`); the strip-and-recompile check rejects any annotation-only edit *deterministically*, and the 4 defects reproduce against the pinned inputs. What is confirmed is the gate’s verdict, not a particular defect count: the *defect-finding* rode unpinned LLM agents from one provider family and is logged rather than re-run (full provenance and replication path, §9).

5 Terrain Archetypes: Where the Truth Lives

The topology above is invariant; the *materials* of each squeeze are not, and the coordinator’s first act on a new instance is to locate where truth lives. We instantiate the three archetypes as four executable, reproducible squeezes (Table 4): **A** tabular analytics and **D** formal proofs in Rocq,

both *transcription* on very different lower bounds; **B** a refund agent on *authored authority*; and **C** an API contract guard on *split planes*. Each ran as a coordinator–worker loop of LLM coding agents, measured end-to-end. Each is a *wiring check* — we seed an internally-consistent defect designed to violate the oracle, and the gate catches it — a wiring check that the construction instantiates on the terrain.

A — transcription (tabular analytics). Upper bound: a machine-parsable metric handbook. Lower bound: a frozen SQL warehouse plus a signed ledger of 24 figures that recompute byte-for-byte. A gross-revenue (coherent-and-wrong) implementer is rejected at Gate B because the exerciser, holding only the handbook, expected net; without that disjoint exerciser the wrong number has nothing to fail against — the no-independent-oracle regime of the test-oracle problem [6]. Scripts under `src/A/` regenerate every figure.

B — authored authority (refund agent). The hardest terrain: no external authority exists, so the lower bound (a frozen REST app plus a signed archive of 6 adjudicated decisions) shares the policy author’s provenance. The honest reading is an *asymmetric* guarantee — the squeeze removes conversational anchoring (a *necessary* condition), but, lacking an external oracle, cannot supply sufficiency; it reduces to author independence (the Fagan/Cleanroom move) plus that anchoring guarantee, and no more. Seeded policy-violating bots [19, 37] are caught by the spec-only exerciser via a REST lockpoint admitting only `REIMBURSE/DENY/ESCALATE`; the genuinely *live* refund result is §6.

C — split planes (API contract guard). Correctness divides across a document plane (OpenAPI schema) and a runtime plane (live server); the characteristic failure is *blending* — compliant docs masking broken runtime. The exerciser derives 5 conformance cases from the schema alone and runs them against the live server [3]; seeded clause-violating servers (auth skip, leaked id, raw traceback) are caught by the no-blend cross-check, which holds the two planes independent rather than letting one stand in for the other.

D — transcription, deductively (Rocq proofs). The link needs no *runnable* oracle: here the hard truth is type-checking. A textbook exercise is transcribed into a formal statement and discharged in Rocq (formerly Coq); the gate is the kernel plus an axiom audit (**Print Assumptions**) rejecting any **Admitted** escape, with a code-blind exerciser writing false mutations. A vacuous or adjacent-property proof is rejected — the same construction over a deductive ground truth, where independence is enforced rather than measured.

5.1 Synthesis: one topology, four materials

The four instances make the paper’s central structural claim concrete: the loop topology is invariant while the *materials* of each squeeze are rebuilt per terrain (Table 4). Across 4 instances over all 3 archetypes the construction instantiates end-to-end and its gate fires on each seeded coherent-and-wrong. We report no barrier-on/off detection rate here: in the one case the off-condition was run with live agents (the pilot of §8) the exerciser caught the defect *regardless* of the barrier, because it derives its acceptance test from the spec, not the code. The barrier would earn its keep only where a judge could otherwise *anchor* to an implementation it sees — and even there its measured outcome effect was null (§8); on spec-derived checks it changes nothing. The cross-instance counts regenerate from each terrain’s evidence (`verify/synthesis.py`), not hand-typed.

Each archetype places the soundness load on a different gate (Table 4): transcription on Gate B + total additivity, authored authority on the coherent-and-wrong guard defended by

	A — transcription	B — authored authority	C — split planes	D — transcription (deductive)
Use case	tabular analytics	autonomous refund agent	API contract guard	formal proofs
Upper bound made of	a metric handbook	an authored refund policy	an API governance manifest	a textbook manifest
Lower bound made of	a signed SQL warehouse + baseline ledger	a REST app + a signed decision archive	an OpenAPI document + a live server	the Rocq kernel + axiom-audited proofs
Dominant failure	stale / over-claimed number	manipulated decision	blended planes (docs mask runtime)	vacuous / wrong theorem
Load-bearing gate	Gate B + total additivity	coherent-and-wrong guard (exerciser independence)	no-blend cross-check	kernel + axiom audit

Table 4: The four executable instances over three terrain archetypes (A and D both transcription): one invariant topology, different materials, a different gate carrying the soundness per terrain. We report no seeded barrier-on/off detection rate (see §5.1).

exerciser independence, split planes on the no-blend cross-check, the deductive instance on the kernel’s axiom audit — one topology shifting its weight to local materials.

6 A Live-Model Failure Mode: Underspecification and Contradiction

The seeded instances above test the machinery on *specified* tasks. The more interesting live failure appears where the upper bound is a genuine *soft* truth the model must resolve — *underspecified* (it leaves cases implicit) or *self-contradictory* (its own clauses, or two authorities it cites, disagree) — the failures capable models do *not* reliably resist. This is the paper’s second live result, beside the real-world defects of Section 4; we state it here and give its measurement apparatus with the evaluation protocol (Section 8).

Manipulation is a wall capable models resist unaided; an *underspecified* upper bound is not. An exploratory probe (LLM sub-agents, weak to strong; logged under `verify/reports/live/`, non-deterministic, so *not* gated) gave each instance only its soft bound on cases it leaves implicit, scored against the executable oracle. A control isolates signal from artifact: with the model allowed to *reason*, three of the four surfaces *vanish* on these *thin* specs — on analytics, the API guard, and proofs the strong model scores perfectly. But thinness, not the terrain, did that work: re-armed as soft truths that carry a defect, analytics and the API guard make capable models err again (*Contradiction, not only silence*, below). Proofs — the deductive instance D — are deliberately *not* re-armed, and the live floor measured below is therefore A, B, and C only: D has no *soft* upper bound to contradict. Its bound is a *formal* specification and its gate a proof checker, so “coherent-and-wrong” on D is a fabricated or vacuous proof, which its gate rejects *deterministically* — the Rocq kernel plus an axiom audit (`Print Assumptions`) and code-blind mutations (§5), the deductive cousin of `lace`’s strip-and-recompile faithfulness check (§4) — a yes/no construction check, not a stochastic divergence with a rate that varies by capability or provider. D’s evidence is the gate firing by construction; it has no capability curve to draw. The refund agent is different: its surface *survives and grows* under reasoning, and across the tiers we

ran it showed *no monotone improvement with capability*, the strongest model diverging as often as the weakest (about a quarter of a balanced gap pool), with three cases missed by *every* tier (one draw per cell; the powered qwen ladder in Figure 3 repeats it). The oracle encodes defaults the prose never states (a delivered order it never rules on), and a capable model fills the gap with its own *coherent* policy that diverges from the answer key. We do not claim the model is *wrong* — the underspecified bound leaves the reading open — but that the squeeze performs *latent-underspecification surfacing*: it exposes a spec gap self-evaluation cannot see and routes it to the policy-owner for adjudication.

A pipeline comparison on the same cases quantifies what that gate earns. Scoring each *shipped* decision against the key: raw generation and self-authored tests ship the coherent-and-wrong on most gap cases; self-critique halves it; a same-context critic and an LLM-judge cut it to about a sixth; only the squeeze’s *executable oracle* ships none. What every model-based check passes, and the oracle alone catches, is the model-*consensus* reading (a damaged or changed-mind item refunded where the authored decider escalates). The nuance: a same-model compliance check already catches the clause-derivable errors, so the gate’s irreducible value is the executable lower bound, not model disjointness. A same-provider cross-model cast confirms this within the family — across every implementer/exerciser tier pairing, same- and same-provider cross-model alike, the consensus errors ship *identically*. A single *cross-provider* model (a 27B Qwen, Alibaba — independent pretraining; logged, not gated) gives a first data point beyond the family: it errs at the same 25% rate as the haiku and opus casts (3/12, stable across repetitions) and shares the blind spot on the pure unstated-default cases (a delivered order; an already-refunded one), so the surface is not Anthropic-specific. But the correlation is only *partial* — the Qwen reads one “manager” case correctly where every Claude tier escalated, and newly caves on a high-value new account. A genuinely independent provider thus decorrelates on part of the surface: direct, if preliminary, evidence for the limit of §9. This is the Knight–Leveson correlation, measured *within one provider family* and probed once across providers; a fully powered, multi-provider version is future work.

Contradiction, not only silence. A soft truth fails the model not only by *silence* (under-specification) but by *self-contradiction*, and contradiction does not wash out with capability. We re-armed the analytics handbook (A) and the API manifest (C) with *authority-vs-authority* defects — two named, real authorities that genuinely disagree: for A, ASC 606 (the US revenue-recognition accounting standard) net-revenue against a binding finance directive reporting gross, a GA4 (Google Analytics 4) “active user” against a product-council directive excluding logins, and a point-in-time data-governance standard against a GDPR (General Data Protection Regulation) Art. 17 erasure directive; for C, RFC 7396 (an IETF Request for Comments; its merge-patch *null deletes* the member) against a vendor “governance override” (Zalando’s guideline) that ignores it. The executable lower bound adjudicates each; we measured how often a live model handed only the contradictory spec lands elsewhere, repeated across a qwen capability ladder (4B/9B/27B; $n=24$ per contested fork) and a three-provider panel (Alibaba **qwen3.6:27b**, Google **gemma4:31b**, Meta **llama4**; logged under **verify/reports/live/**, not gated).

Figure 3 is the result. On the forks where *both* authorities are genuinely legitimate the error is a *floor*: the GA4 **active** fork (and the unstated-default refund on B) is missed at near-100% at *every* size, and the three-provider panel reproduces this identically (24/24, 24/24, 8/8 across all three — three providers on constructed cases, not invariance in a broader sense).

Scaling makes the contradiction worse. The GDPR-erasure *survivorship* fork is the sharper finding: its error *rises* with capability ($0.54 \rightarrow 0.83 \rightarrow 1.00$ across the qwen ladder), the larger models more confidently following the erasure override into the wrong answer. So on a genuinely-balanced contradiction, capability is not merely unhelpful but actively harmful. Capability settles only the *decidable* conflicts (a sloppy **net/gross** directive, an

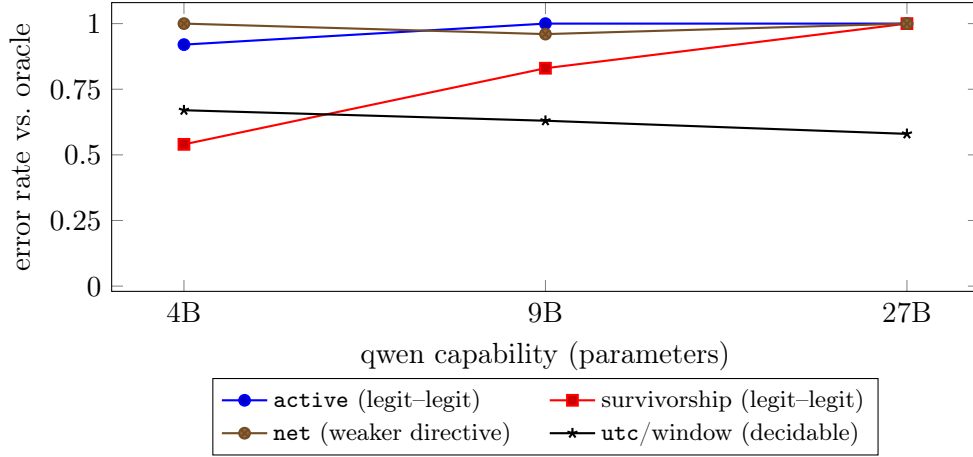


Figure 3: On a genuinely-balanced contradiction, scaling makes the error *worse*: the GDPR-erasure *survivorship* fork rises $0.54 \rightarrow 1.00$ with model capability — the larger model follows one authority off the cliff more confidently. Error rate vs. capability on contradiction instance A, across the qwen ladder (4B/9B/27B; $n=24$ per fork; the two 27B generations pooled at 27B). The other genuinely-balanced fork (**active**) is a near-100% floor throughout; the decidable **utc** fork stays low. At flagship scale the three-provider panel (Alibaba/Google/Meta) reproduces the balanced-fork floor at 100%; on the weaker-side forks the providers spread out (gemma most override-compliant, llama least). Underlying counts in `verify/reports/live/`. All values are point estimates ($n=24$ per fork per size) with no confidence intervals; the replicable target is the qualitative trend—the *survivorship* fork rising with capability, the genuinely-balanced floor forks staying near 100%—not the exact decimals.

inconsistent window), and there genuine model diversity *spreads out* rather than agreeing (on **net**, llama resists 16/24 where qwen and gemma err 24/24; on the patch-null override gemma 16/16 > qwen 12/16 > llama 7/16). On the genuine contradictions it is powerless and the disjoint executable lower bound is the only adjudicator. (On instance B the floor fork `delivered_default` “diverges” from the authored archive rather than being “wrong”: B’s archive shares the policy-owner’s provenance, so this is underspecification *surfaced*, not detection against an independent oracle, which B lacks.) These are deliberately constructed defects — an existence demonstration of the failure mode and its capability profile, not a frequency over real specs.

Replication. A replicator needs the `build_cases()` probes (`src/{A,B,C}/ground-truth/`) scored against each instance’s executable oracle. The confirmed target is the *floor*, not the per-draw decimal: the GA4 active-user contradiction and the unstated-default refund are near-100% across every model we ran (Claude haiku-opus, qwen 4B–27B, gemma, llama), while the GDPR-erasure survivorship fork is near-100% at flagship scale and rises with capability (lower on the smallest qwen). The exact rate is non-deterministic and logged, not gated (full path, §9).

7 The Reflexive Case Study: This Paper as a Squeeze Loop

This manuscript was produced under the strategy it describes: its upper bound is the peer-reviewed literature (each source recorded before it may be cited), its lower bound the 4 executable instances and their harnesses, and *done* is the claim ledger re-derived, not the authors’ satisfaction. The point of interest is what that process caught in *our own* claims before the paper shipped: 8 coherent-and-wrong or over-claim defects (4 on the literature plane, 4 on the evidence plane) — a comparative overstated, a finding misattributed, a figure crediting the

wrong source, an abridged quotation shown as verbatim — each fluent and correct-looking, each found by re-deriving the claim from its source, not by re-reading the prose (all 8 enumerated in Appendix A). This is the global story turned inward: coherent-and-wrong is not a property of implementations alone; soft-side editorial claims fail the same way, and only re-derivation from an independent source catches them.

The honest limit is in the strategy’s own terms: a single team is not independent of itself, so the gate’s independence was only *approximated* — a disjoint same-provider sub-agent, a cross-provider framing judge, and the human peer reviewer as the mandatory terminus that closes the un-squeezed Gate A (who caught 1 defect our own gates missed). It is an $n=1$ illustration of the mechanism catching its target, not a controlled result; the approximation’s mechanics are in `paper-impl.md`.

8 Proposed Evaluation Protocol

This section is a protocol proposal with mostly-null pilots, not a completed evaluation. The strategy’s efficacy is *non-linear*, so these null pilots are a *boundary condition*, not a shortfall: on small, well-specified tasks the per-implementation error base rate is near zero, leaving the barrier nothing to catch and no marginal value to add; it becomes strictly necessary only on *contested* soft specs — underspecified or self-contradictory — where capable models diverge and added capability does not fix it (§6). The deterministic mechanism is established on the four executable instances (Section 5.1), but it has not been evaluated under *controlled*, *comparative* conditions. This section fixes the hypotheses and measurements we expect to matter, operationalizes them as a runnable harness (`verify/eval_protocol.py`), and reports what we have so far been able to run. The four hypotheses are: **H1**, disjointness lowers the coherent-and-wrong rate versus shared context; **H2**, physical barriers reduce evidence–implementation coupling and catch more seeded defects; **H3**, the Gate-A amendment rate predicts the downstream gap rate; and **H4**, protocol overhead amortizes on long regression horizons. The supporting evidence we have today is exploratory: one *live* result (instance B’s underspecified cases diverge from the authored archive in a way added capability does not fix, surfaced only by the executable lower bound — coherent-and-wrong in the *surfacing* sense of §6, not adjudicated against an independent oracle, which B lacks; below) and one *reflexive* illustration (this manuscript had 8 coherent-and-wrong or over-claim defects caught before submission, 1 only by external review; Section 7) — existence proofs that the gate catches the target failure, not rates.

Protocol status. Of the harness’s 4 hypotheses, 1 (H2) is *partially* demonstrated on the four instances as a consistency check: its seeded-detection half shows only that the gate fires on each terrain’s seeded coherent-and-wrong (a wiring check; we report no barrier-on/off detection rate, §5.1), while its *coupling* half is only illustrative and the barrier-off delta remains future work. The remaining 3 (H1, H3, H4) and 3 comparative configurations are wired as explicit stubs reporting their prerequisites (a real LLM-agent pipeline, a matched-difficulty suite, cross-model casts, repetitions, a significance test); no numbers are fabricated. The nulls confirm only that the seeded tasks are too easy to elicit coherent-and-wrong; the live evidence below comes from the contested regime instead.

What we have run. The comparative configurations were executed as pilots with LLM sub-agents, and all return the same null. With a seeded coherent-and-wrong implementation, the exerciser caught the defect on all 4 tasks whether shown the specification alone or also the code: it tested against the spec, not the code. Across self-authorship, a weaker model, and a pre-committed 24-implementation study over three tiers on harder tasks, the implementers produced almost no bugs (0, 0, 0; McNemar $p = 1.000$ on the last — vacuous, no discordant pairs), so no self-authored defect existed to miss. The obstacle is not statistical power but a near-zero

per-implementation error rate: current models reliably get small, well-specified functions right. We report these nulls rather than hunt task space for an effect. The strategy’s value is in any case a *guarantee* — the barrier removes the *possibility* of anchoring; these results bound only how often it is realized.

A comparative baseline on defective soft specs. The near-zero rate is a property of *thin, well-specified* tasks, not of the strategy: a task with one plausible reading leaves a capable model nothing to get coherently wrong. When the upper bound is instead a genuine soft truth carrying a defect — an underspecified gap (B) or an authority-vs-authority contradiction (A and C, §6) — live capable models do err, at a nonzero rate only the executable lower bound catches. Measured across Claude tiers (weak/mid/strong; logged, not gated), the per-instance error against the oracle was, on the contradiction-armed specs at *five* draws per case, A 46/75, 25/75, 40/75 (non-monotone, with the two both-authorities-legitimate forks wrong at every tier) and C a flat 11/90, 12/90, 10/90; and on the underspecified refund policy about 3/12 per tier (§6); capability brings no monotone gain on any of them, and on A’s both-authorities-legitimate forks no gain at all. This is the comparative signal the seeded near-zero pilots could not surface. It is an existence demonstration on *constructed* defects, not a benchmark rate over real specs; a genuinely multi-provider version remains future work.

Toward a real-bug study. A controlled comparison against SWE-bench-Lite [24] is wired as a future path; its effect size awaits the per-repository Docker images and patch-generating endpoint absent from this build.

Mechanism demonstrations (deterministic, not learning results). Three pieces of machinery are wired and shown causal on the executable instances, none a live-model claim. (i) A *level-up* apparatus enriches the thin upper bound that makes capable models right by default: a difficulty ladder per terrain plus a pool of graded-subtlety tasks, with the same detection primitive throughout (an independent re-derivation flagged when a candidate diverges), exposing a genuine nonzero error surface against a fixed *weak* implementer; random sampling of the pool also restores a genuine per-cycle trial. (ii) The squeeze’s “get caught → consolidate” move is wired into each instance: the deciding agent starts from scratch, the fixed independent exerciser catches each coherent-and-wrong move, and consolidation removes that error class — causal (re-playing final cycles against an empty store remakes the errors) and deterministic (byte-identical on re-run); the deductive instance shows the bounded case, where a residual conceptual-leap wall persists. (iii) **Gate S** (Section 3.3) *squeezes* the accumulated skills themselves: a monitor holding each instance’s upper bound and executable oracle — a different evidence base than the deciding agent’s caught errors — audits them in a recomputable check, repairing an over-reaching “ignore-it” skill by a narrowing carve-out that defers to the upper bound. These establish only that the machinery is wired and causal — a deterministic loop eliminates exactly the errors it consolidates against; they are programmatic learners over fixed catalogs, not live models, so a live model’s learning curve remains open. Full per-instance counts are in the supplementary harness.

What a full evaluation would add (not run). A powered, comparative protocol — squeeze-compliant versus shared-context pipelines at matched difficulty, with defect-escape and coherent-and-wrong rates as outcomes, the stabilizers ablated one at a time, and a human audit of Gate C verdicts — is specified in the supplementary harness but not run here. The natural-bug route met a near-zero error rate, and the controlled, multi-provider version needs infrastructure we did not have (per-repository Docker images, a patch-generating endpoint, and casts spanning genuinely independent providers). We therefore present the *powered* comparison as specified-but-unrun — the exploratory pieces above were run.

Isolating the barrier: a clean negative. Does removing the implementation from a judge’s context change an *outcome*? We ran the experiment that targets this LLM-specific novelty directly, fixing the judging mode (write the acceptance figure; accept iff a test asserting it passes) and varying only the test-writer’s context across three arms: spec *alone*; spec *and* the defective code; spec plus a length-matched *unrelated* file (placebo). On the well-specified gross-vs-net seed (§5), across weak/mid/strong tiers at 12 draws per cell, a draw *escapes* if its test ratifies the defective code. Escape was 0/12 in *every* cell, all tiers, all arms: every judge, *including* those shown the defective code, derived the spec’s figure and caught the defect (several named the code buggy). The barrier moved nothing. We then ran the arms where it *should* bind — an *underspecified* spec (where the dominant response was the squeeze-correct one, *escalate to the policy-owner*, not anchoring) and a *persuasion* soft-side arm (a binding policy, a clear violation, a compliance-judge shown the agent’s fluent self-justification) — and escape was 0/12 there too. Across all arms (324 draws) the barrier’s outcome-effect is undemonstrated, for one reason: given an *explicit* authority a capable model follows it (or escalates the gap) regardless of what artifact-side material sits in its context.

A weaker, cross-provider ladder (qwen 4B/9B/27B, all passing the control probes) tested the one condition the Claude tiers could not rule out — a genuinely weaker judge — and escape stayed 0/10. Across 504 draws over three regimes, two provider families, and the 4B–Opus range, the barrier’s outcome-effect never appeared. We do not read this as the barrier being useless: the anchoring and sycophancy failures it targets are documented (§2). But the regime where they would bind — a judge following the *artifact* over an *explicit* authority — is one a clean isolation excludes: a model that cannot hold the authority fails the control probes (incompetence, not coherent-and-wrong), and the experiment must *supply* an explicit authority, the very condition under which a competent judge does not need the barrier. We endorse interpretation (a): for a competent model—the only kind whose verdict we would trust—the context barrier is structurally present but empirically redundant, because such a model follows the explicit authority over the artifact whether or not it can inspect the implementation. We do *not* claim the barrier is inert for weak judges, where anchoring might bind; we claim it does not change outcomes in the regime that matters. The executable oracle and the authority, not the context barrier, carry the load in every condition we could *cleanly* test.

9 Limitations

We close by bounding what the strategy does *not* establish, worst first. Each names a specific place where the link between the soft authority and the executable lower bound is still incomplete — on its hard side, on its soft side, or in how far our evidence reaches.

The harder failure is the one the oracle cannot see. The construction is strongest against an artifact that *violates* the executable lower bound and weakest against its dual — an artifact that *satisfies* the hard check while betraying the soft intent (§3). That dual passes the oracle by definition, so catching it falls entirely to Gate A, the editorial judgment that is un-squeezed within a single loop and closed only by nesting terminating in a human (§3.3). The *by-construction* guarantee therefore covers the catchable half — violations of the hard bound — and not the half often more dangerous in practice: a fluent, oracle-passing artifact that is quietly wrong about what was wanted. On that half we have disjoint authority terminating in human review, not a mechanical guarantee. This bounds the whole claim.

Cost and latency. The protocol multiplies agent invocations and adds documentary overhead. The *structural* cost is fixed by the cast, not the task: each item is handled by a coordinator, an implementer, an independently-authored exerciser, and the gates, with a plan document and any gap documents — structurally a low single-digit multiple of a single-agent pass (on the order of a handful of role-invocations per item — the coordinator’s delegation and gate approval, the implementer, and the exerciser — plus two short documents, before any retries), paid per

item. What that costs in *tokens and wall-clock* needs per-engagement telemetry we have not yet instrumented. Hypothesis H4 conjectures, but does not yet show, that this amortizes on engagements with long regression horizons and not on one-shot tasks. Because current models rarely produce coherent-and-wrong on the small, well-specified tasks exercised here, the base rate of the failure the protocol prevents is itself unmeasured; the external application of Section 4 shows such defects *do* occur in production code, without measuring their rate. The cost–benefit case rests for now on the literature of Section 2 and the deferred real-bug study (Section 8).

Shared priors. Context barriers remove *conversational* coupling between agents, not the correlation induced by shared pretraining — the Knight–Leveson lesson [27] transposed. We *measured* it: same- and same-provider cross-model casts ship the identical consensus errors, so disjoint *models* do not catch them; only the executable lower bound does (§6). The Knight–Leveson claim proper — that correlated error survives *genuine* model diversity — cannot be settled from within-family data, so we replicated across a qwen capability ladder (4B/9B/27B) and three genuinely-independent lineages (Alibaba/Google/Meta): on the genuinely-balanced contradiction forks the floor reproduces at 100% across all three, while on the weaker, decidable forks the providers *spread out* — diversity decorrelates only where an oracle is least needed (Figure 3). This is begun, not powered: three lineages, not dozens (DeepSeek and OpenAI gpt-oss could not be run at power on our host). We state the dependency plainly: where an executable oracle exists the gate rests on the lower bound, which holds regardless of model correlation; the *distinctive* claim over prover–verifier games — that disjointness of evidence buys independence on oracle-free terrains where only model casts disagree — rests on that cross-provider replication, which points the predicted way but is not yet powered. We claim the within-family results and this preliminary multi-provider corroboration, and no more.

The coordinator is a single point of judgment at one level. Gates A and C concentrate editorial authority; a negligent coordinator degrades the loop even though it cannot silently corrupt artifacts, since it never touches source. Nesting is the construction’s answer (§3): an outer squeeze holds a disjoint (U, L) pair over the inner coordinator, so no level’s editorial judge is the last word — the recursion *must* terminate at a human (an external reviewer) who squeezes the top coordinator; automation alone never closes it. This closes the soft-side hole by *disjoint authority* rather than by construction; what it does not buy is a mechanical, judge-free guarantee on the soft side, which only the hard executable bound has. **An executable lower bound is required:** where none exists, the first work item must build one (probes, goldens), and domains where no cheap runnable ground truth can be constructed are outside the strategy’s current reach.

The lower bound can itself be coherent-and-wrong. The gate rests on an executable ground truth, but a flaky or mistaken oracle that happens to agree with wrong code is coherent-and-wrong at the hard bound itself. Total additivity, determinism re-runs, and a signed baseline mitigate but do not eliminate this, since a test wrong in the same direction as the code escapes them. The risk is least where the lower bound is a proof kernel, greatest where it is authored tests, which then deserve the same independence scrutiny as the implementation.

Provenance of the evidence. The four executable instances are constructed, deterministic, and small, built by one team operating LLM coding agents: reproducible end-to-end but not a controlled comparison, so the generalization claim rests on the evaluation of Section 8, operationalized and piloted but not yet powered (the pilots met a near-zero natural error rate, leaving the effect size to a real-bug study). The external *lace* application (Section 4) adds evidence on code the team did not author, where the strategy found 4 natural defects — a bug-finding case study without a matched baseline, nor a full verification of that codebase. One reproducibility gap is our own: we pin `rustc` 1.96.0 for the *lace* reproductions, but the LLM agents are Anthropic Claude models from a single provider family rather than per-run pinned checkpoints. For the no-improvement-with-capability observation we report a within-family tier gradient, with exact per-run snapshots in the exploratory logs (`verify/reports/live/`); a

cross-provider, dated-checkpoint replication is owed. There is an honest irony to confront: the numbers our deterministic harness regenerates byte-for-byte are the *seeded* consistency checks — the part we explicitly do *not* rest load on — while the two load-bearing findings (§4, §6) are the non-deterministic, unpinned ones. Reproducibility-by-construction holds for the wiring; the evidence that earns the architecture rests, for now, on logged transcripts rather than a re-run. *What a replication needs*, consolidated: for `lace`, `rustc` 1.96.0 plus the Creusot toolchain and the faithfulness gate (`verify/`), where the strip-and-recompile check rejects any annotation-only edit *deterministically* and the 4 defects reproduce against the pinned inputs; for the contradiction/underspecification finding, the `build_cases()` probes (`src/{A,B,C}/ground-truth/`) scored against each instance’s executable oracle, on the instruction-tuned models we ran (below). The reproducible target is the *floor*, not the per-draw decimal: the GA4 active-user contradiction and the unstated-default refund were missed at near-100% by every tier and provider we ran (Claude haiku-opus; qwen 4B–27B; gemma; llama), while the GDPR-erasure survivorship fork was near-100% at flagship scale but *deepened* with capability (lower on the smallest qwen). So a replicating team should expect those forks missed at a high rate — flat with capability for active and the refund default, rising with it for survivorship — not a particular number. The non-determinism we cannot pin is the exact rate; the qualitative floor reproduced on every model we tried.

Skill accumulation is shown for programmatic learners, not live models. The caught-then-consolidate result (Section 8) is established with *deterministic programmatic learners over fixed catalogs*, not live LLMs; a live model’s learning curve, and how the deductive instance’s deferred conceptual-leap wall is crossed, remain open.

10 Conclusion

The squeeze loop strategy is a small number of decisions applied without exception. Hold every actor between an authority it must answer to and a ground truth it cannot touch; make those authority pairs *different* for every actor (the *disjointness principle*), so the intersection of all constraints is the correct deliverable while no single evidence base suffices; route all motion through editorially approved plans and gap documents; and let machine gates define done, never an agent’s report. What actually *catches* a coherent-and-wrong artifact, in every condition we could measure, is the executable lower bound — an independent oracle that runs and returns a verdict, not a smarter model, not more agents, and not hiding the code from the judge. Disjointness of evidence carries the load on the one terrain where no such oracle exists (Archetype B), and elsewhere is the precondition that catch builds on. Four executable instances over three epistemic terrains kept this topology fixed and rebuilt the squeeze from local materials each time: the strategy’s portable core, shown by construction. Applied unchanged to an external UEFI bootloader the authors did not write (Canonical’s `lace`), the loop proved 25 input-facing functions robust and surfaced 4 real security defects — with the squeeze-specific guarantee that the proofs are *trustworthy by construction*, a fabricated proof unable to survive the faithfulness gate (a case study, not the controlled comparison still owed, Section 8). The second live finding earns the architecture: on an underspecified or self-contradictory policy a capable agent supplies its own coherent reading that diverges from the answer key, a divergence that showed no *monotone* improvement as capability rose and that same- and same-provider cross-model casts ship *identically* — only the executable lower bound surfaces it (Section 6). Whether the gates shift outcomes against the natural error rate of strong models is left as a wired but underpowered study, having declined to manufacture an effect the data did not show. Applied to its own construction, the strategy caught 8 coherent-and-wrong or over-claim defects in this paper’s own claims before submission (1 only by external review) — a small $n=1$ illustration, not a controlled result (Section 7). The compact statement we end on: *give every agent a truth it must answer to and a truth it cannot touch, make those truths different for every agent, and let convergence*

A The coherent-and-wrong and over-claim defects the process caught

Section 7 reports that the paper’s own process caught 8 coherent-and-wrong or over-claim defects in its claims before submission (1 only by external review). We enumerate them here so the claim is checkable rather than taken on trust. Each is recorded in `verify/manuscript_defects.tsv` and was found by re-deriving the claim from its source or artifact, not by re-reading the prose: 4 on the literature plane, 4 on the evidence plane.

	Plane	Claim as written (coherent-and-wrong)	Correction
D1	literature	a survey comparison stated as “outperforms” more strongly than the cited source asserts	softened to the source’s hedged claim
D2	literature	full reasoning shared across debate rounds attributed to Irving et al. [22]	reattributed to Du et al. [14], the only realization it holds of
D3	literature	Constitutional AI described as sharing weights, unscoped	scoped to its self-critique stage
D7	literature	Clover’s “passing all checks implies correctness” stated as fact	restated as the authors’ hypothesis (caught at full-text read)
D4	evidence	a figure attributed Gate C to a guide that has only Gates A–B	provenance corrected
D5	evidence	“all three guides end with this sentence” — the three closing sentences differ	claim removed
D6	evidence	an abridged agent transcript shown in a verbatim environment	marked as abridged
D8*	evidence	a figure caption called Gate A machine-checked; Gate A is editorial judgment	caption corrected

Table 5: The 8 coherent-and-wrong or over-claim defects the process caught in the paper’s own claims before submission. * D8 is the 1 the internal gates missed — a peer reviewer caught it, the one party with true context independence (Section 7).

B Anti-collapse stabilizers

A *collapse* is any event in which an actor relieves its own squeeze — severing, at that node, the link between the soft authority it must answer to and the executable ground truth it cannot touch. The four executable instances converged on 13 rules that block collapse structurally (Table 6 maps collapse modes to the rule that blocks each). These are *operational lore* distilled from running the loop, not theorems — a practitioner checklist, not part of the formal apparatus of Section 3.

1. **Forbidden moves are load-bearing.** Each role’s negative constraints (never edit the drivers, never read the diff, never implement a fix, never touch source) keep the squeeze from collapsing: any actor allowed to cross roles can relieve its own pressure.

2. **Information barriers are physical, not honorary.** Separation is enforced by what is in the delegation prompt (fresh context, surface and acceptance lines only), not by asking an agent to refrain.
3. **Done is gate-defined, never self-declared.** Done is machine gates passing, witnessed by independently authored evidence; self-assessment is never evidence.
4. **Conservation laws bound the blast radius.** A standing invariant (total additivity; standing proof counts; determinism re-runs) is rerun after *every* item, turning regression into a definitional impossibility.
5. **Surprise is routed, never absorbed.** A surprising probe verdict or failing acceptance line becomes a gap document; a local workaround silently desynchronizes the workflow's model of reality.
6. **Pin reality before building on it.** One claim per probe, one minimal experiment, one recorded verdict; feasibility proven on a minimal instance before machinery is built.
7. **Specify the implicit base before extending it.** Transcribe existing de facto behaviour into the references before adding anything; verify by independent prediction.
8. **Editorial approval gates the plan, before code.** The approval must amend, cut, or sharpen; a rubber stamp makes the coordinator's upper bound fictional. No source edit from a DRAFT.
9. **Honest scope is a deliverable.** Explicit NOT-claims and the classification of every escape hatch make what was *not* proven as legible as what was.
10. **Strictness has a safe direction.** The implementation may be stricter than the authority, never weaker; divergence-by-strictness is legitimate, divergence-by-weakness is a bug.
11. **Sequence by mechanism reuse; parallelize only across independence.** Each item rides machinery the previous one hardened; chains are parallel only when they share no evidence base and no deliverable.
12. **Traceability is itself an acceptance criterion.** Every change traces to an approved plan, every plan to the gap or authority section that motivated it.
13. **Where authority is authored, independence is the soundness argument.** With no external spec to diff against, the only defense against coherent-and-wrong is that the property's author and exerciser never saw the implementation; the design must name this (Gate C) and protect it.

C SL-1.0 diagrams for the Creusot application

The two nested loops of the Creusot application (Section 4) are shown here, rendered from their SL-1.0 registries by the same tool that draws every diagram in this paper. They are placed in the appendix, rotated to full page, because the renders are dense; the repository SVGs (`docs/img/creusot-*.svg`) are the lossless, zoomable form. Figure 4 is the per-file object loop; Figure 5 is the crate-level loop that drives and monitors it.

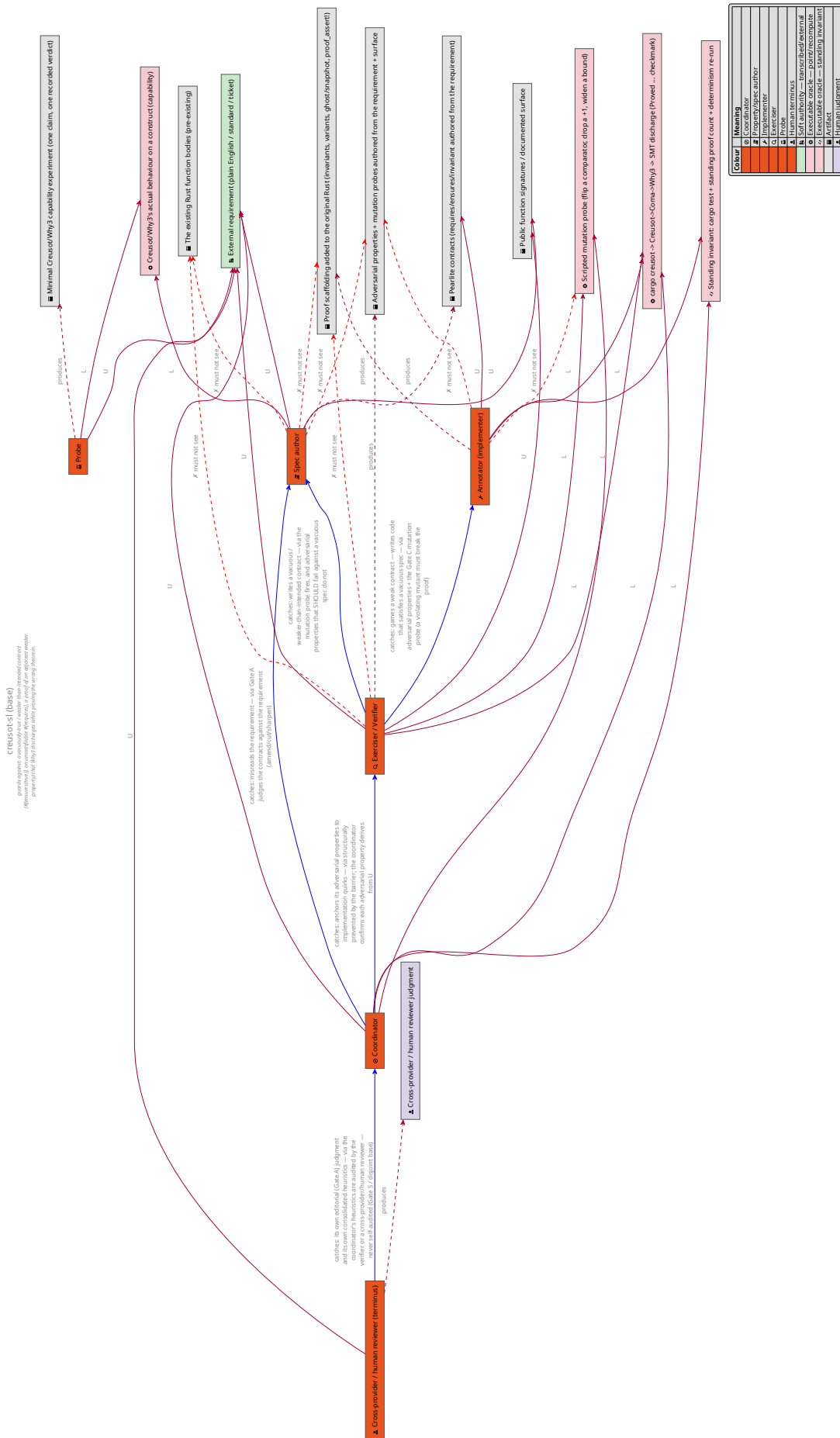


Figure 4: The **creusot-s1** base loop (one Rust file), rendered from its SL-I.0 registry. The annotator (the implementer recast) is squeezed between the Pearlite requirement above and **cargo creusot** discharge plus tests plus a mutation probe below; the exerciser is barriered (**must-not-see**) from the annotated source and judges only by re-running. This is the object loop the monitor of Figure 5 drives once per file. (The figure is generated by the same renderer as the constructed instances; the SVG in the repository is the legible, zoomable form.)

Collapse mode	Symptom	Blocked by
Self-judging	implementer edits or authors its own acceptance; tests pass suspiciously fast	1, 2
Shared evidence	two actors hold identical (U, L) pairs; an error propagates unchecked	(C1) audit
Honorary barrier	internals present in the judge’s context	2
Self-declared done	“the agent reported success” ends the item	3
Absorbed surprise	local workaround; the loop’s model of reality drifts	5
Rubber-stamp approval	plans approved verbatim, instantly	8
Coherent-and-wrong	a valid result proving an adjacent, weaker property	Gate C; 13 where authored
Weakened clause	“it proves now” after the obligation was quietly softened	10
Regression by rationalization	“that diff is expected”	4
Unpinned reality	design built on assumed tool behaviour	6
Unspecified base	features grown on implicit behaviour	7
Scope blur	silent over-claiming; unclassified escape hatches	9
Unsequenced parallelism	parallel chains share an evidence base or deliverable	11
Orphan change	an edit with no plan, no gap, no authority behind it	12

Table 6: Collapse modes and the stabilizers that block them. Numbers refer to the list above; (C1) is the disjointness audit of Definition 2.

References

- [1] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016.
- [2] Cem Anil, Guodong Zhang, Yuhuai Wu, and Roger B. Grosse. Learning to give checkable answers with prover-verifier games. *arXiv preprint arXiv:2108.12099*, 2021.
- [3] Vaggelis Atlidakis, Patrice Godefroid, and Marina Polishchuk. RESTler: Stateful REST API fuzzing. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, pages 748–758. IEEE Press, 2019. doi: 10.1109/ICSE.2019.00083.
- [4] Algirdas Avižienis. The N-version approach to fault-tolerant software. *IEEE Transactions on Software Engineering*, SE-11(12):1491–1501, 1985.
- [5] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- [6] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5): 507–525, 2015. doi: 10.1109/TSE.2014.2372785.
- [7] Kent Beck. *Test-Driven Development: By Example*. Addison-Wesley, 2003.
- [8] Samuel R. Bowman, Jeeyoon Hyun, Ethan Perez, Edwin Chen, Craig Pettit, Scott Heiner, Kamilė Lukošiuūtė, Amanda Askell, Andy Jones, Anna Chen, et al. Measuring progress on scalable oversight for large language models. *arXiv preprint arXiv:2211.03540*, 2022.
- [9] Collin Burns, Pavel Izmailov, Jan Hendrik Kirchner, Bowen Baker, Leo Gao, Leopold Aschenbrenner, Yining Chen, Adrien Ecoffet, Manas Joglekar, Jan Leike, Ilya Sutskever, and Jeff Wu. Weak-to-strong generalization: Eliciting strong capabilities with weak supervi-

- sion. In *International Conference on Machine Learning (ICML)*, pages 4971–5012, 2024. arXiv:2312.09390.
- [10] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2025. arXiv:2503.13657.
- [11] David D. Clark and David R. Wilson. A comparison of commercial and military computer security policies. In *Proceedings of the IEEE Symposium on Security and Privacy*, pages 184–194, 1987.
- [12] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978. doi: 10.1109/C-M.1978.218136.
- [13] Xavier Denis, Jacques-Henri Jourdan, and Claude Marché. Creusot: A foundry for the deductive verification of Rust programs. In *Formal Methods and Software Engineering (ICFEM 2022)*, volume 13478 of *Lecture Notes in Computer Science*, pages 90–105. Springer, 2022.
- [14] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *International Conference on Machine Learning (ICML)*, pages 11733–11763, 2024. arXiv:2305.14325.
- [15] Michael E. Fagan. Design and code inspections to reduce errors in program development. *IBM Systems Journal*, 15(3):182–211, 1976.
- [16] Jean-Christophe Filliâtre and Andrei Paskevich. Why3 — where programs meet provers. In *Programming Languages and Systems (ESOP)*, volume 7792 of *Lecture Notes in Computer Science*, pages 125–128. Springer, 2013.
- [17] Mario Gleirscher, Jaco van de Pol, and Jim Woodcock. A manifesto for applicable formal methods. *Software and Systems Modeling*, 2023. doi: 10.1007/s10270-023-01124-2.
- [18] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2305.11738.
- [19] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISeC)*, pages 79–90, 2023. doi: 10.1145/3605764.3623985. arXiv:2302.12173.
- [20] Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.00352.
- [21] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.01798.
- [22] Geoffrey Irving, Paul Christiano, and Dario Amodei. AI safety via debate. *arXiv preprint arXiv:1805.00899*, 2018.
- [23] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011. doi: 10.1109/TSE.2010.62.

- [24] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.
- [25] Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Mudit Verma, Kaya Stechly, Siddhant Bhambri, Lucas Saldyt, and Anil Murthy. LLMs can’t plan, but can help planning in LLM-Modulo frameworks. In *International Conference on Machine Learning (ICML)*, pages 22895–22907, 2024. arXiv:2402.01817.
- [26] Jan Hendrik Kirchner, Yining Chen, Harri Edwards, Jan Leike, Nat McAleese, and Yuri Burda. Prover-verifier games improve legibility of LLM outputs. *arXiv preprint arXiv:2407.13692*, 2024.
- [27] John C. Knight and Nancy G. Leveson. An experimental evaluation of the assumption of independence in multiversion programming. *IEEE Transactions on Software Engineering*, SE-12(1):96–109, 1986.
- [28] Victoria Krakovna, Jonathan Uesato, Vladimir Mikulik, Matthew Rahtz, Tom Everitt, Ramana Kumar, Zac Kenton, Jan Leike, and Shane Legg. Specification gaming: The flip side of AI ingenuity. DeepMind Blog, 2020.
- [29] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for “mind” exploration of large language model society. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.17760.
- [30] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2305.20050.
- [31] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegraffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.17651.
- [32] David Manheim and Scott Garrabrant. Categorizing variants of Goodhart’s law. *arXiv preprint arXiv:1803.04585*, 2018.
- [33] William M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998.
- [34] Bertrand Meyer. Applying “design by contract”. *Computer*, 25(10):40–51, 1992.
- [35] Harlan D. Mills, Michael Dyer, and Richard C. Linger. Cleanroom software engineering. *IEEE Software*, 4(5):19–25, 1987.
- [36] Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2404.13076.
- [37] Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. In *NeurIPS 2022 Workshop on Machine Learning Safety*, 2022. arXiv:2211.09527.
- [38] Karl R. Popper. *The Logic of Scientific Discovery*. Hutchinson, 1959.
- [39] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 15174–15186, 2024. arXiv:2307.07924.
- [40] Tal Ridnik, Dedy Kredo, and Itamar Friedman. Code generation with AlphaCodium: From prompt engineering to flow engineering. *arXiv preprint arXiv:2401.08500*, 2024.

- [41] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.
- [42] Mrinank Sharma, Meg Tong, Tomasz Korbak, David Duvenaud, Amanda Askell, Samuel R. Bowman, Newton Cheng, Esin Durmus, Zac Hatfield-Dodds, Scott R. Johnston, et al. Towards understanding sycophancy in language models. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.13548.
- [43] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.11366.
- [44] Chuyue Sun, Ying Sheng, Oded Padon, and Clark Barrett. Clover: Closed-loop verifiable code generation. In *International Symposium on AI Verification (SAIV)*, Lecture Notes in Computer Science, pages 134–155. Springer, 2024. arXiv:2310.17807.
- [45] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research (TMLR)*, 2024. arXiv:2305.16291.
- [46] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *Conference on Language Modeling (COLM)*, 2024. arXiv:2308.08155.
- [47] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.15793.
- [48] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.
- [49] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023. arXiv:2306.05685.